Tech Nation Visa — Global Talent (Exceptional Promise)

Criteria: Optional Criteria 3 — Significant Technical, Commercial, or Entrepreneurial Contributions

Applicant: Odeyemi Olusegun Israel

OC3-1: Machine Learning Pipeline & Model Validation

Executive Summary: I designed, trained, and deployed the AI fraud detection engine used by AMTTP. My role covered every stage of the ML lifecycle: feature engineering, training, evaluation, knowledge distillation, and production deployment. The system runs as a nine-service compliance orchestrator combining gradient-boosted ML, graph-network analysis, and deterministic AML policy evaluation — designed for CPU-only deployment with no specialist hardware requirement.

Table: OC3-1 Evidence Summary — ML Pipeline, Architecture and Validation Claims

Table: OC3-1 Evidence Summary

1. Model Architecture and Results

The AMTTP compliance engine is built as a composite decision stack, not a standalone ML classifier. The teacher stack combines XGBoost and LightGBM base learners (5-fold out-of-fold), a beta-VAE feature encoder for latent representation, a LogisticRegressionCV stacking meta-learner, Memgraph graph analysis (PageRank, shortest-path sanctions distance, degree centrality, community detection), and deterministic AML policy rule evaluation into a single orchestrated compliance framework. Student models are distilled from this composite teacher system to enable CPU-only deployment while preserving most operational performance. The production student model is XGBoost, augmented with 64 latent features and 3 anomaly metrics from the beta-VAE encoder (160 features total). XGBoost was selected over the full blended ensemble after validation, achieving very high ROC-AUC on the evaluated benchmark (exact figures in Table 1). This reflects the quality of the composite teacher signal rather than any single isolated component.

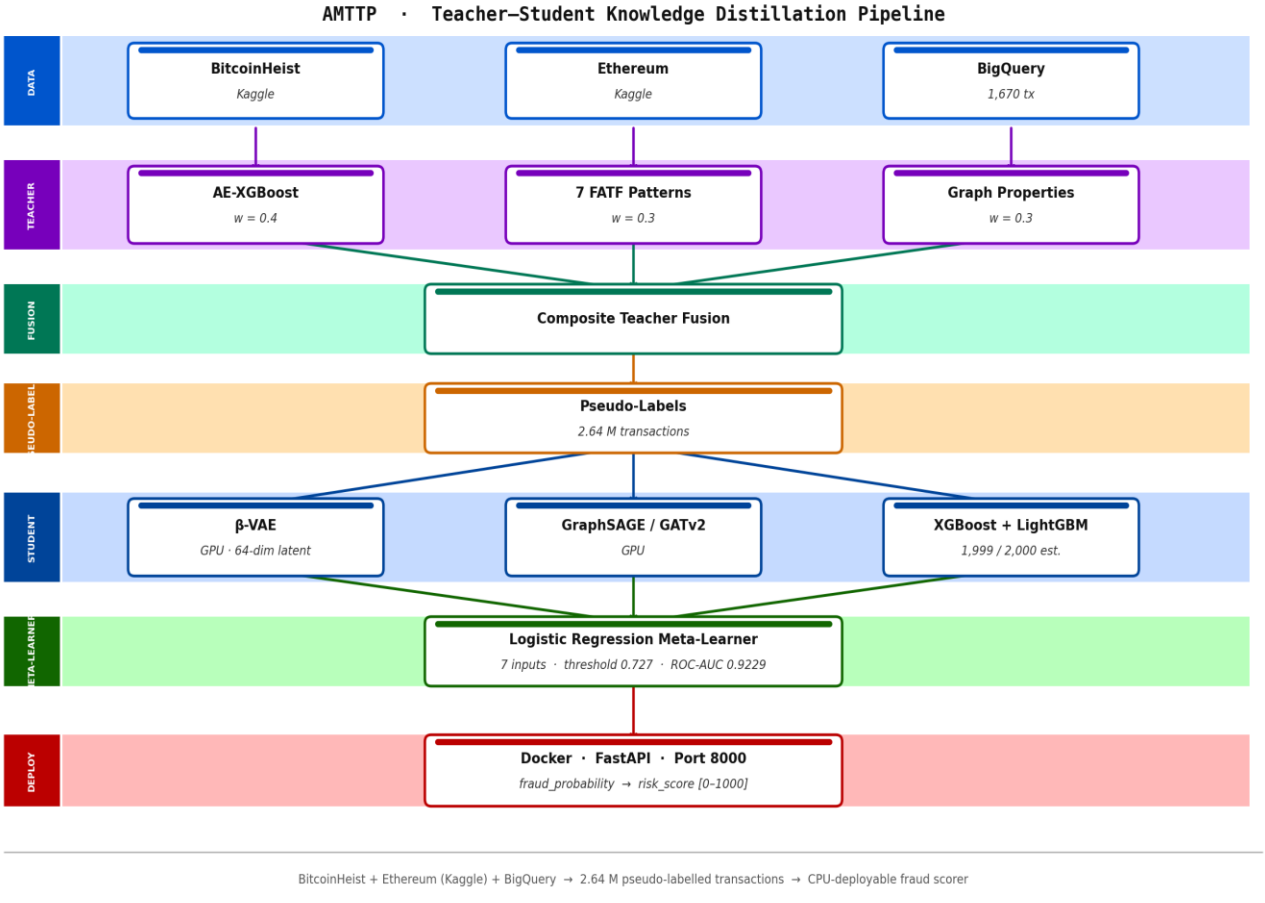

Figure A: AMTTP ML pipeline — teacher–student knowledge distillation architecture.

Feature Importance Implementation (source: ml/Automation/risk_engine/integration_service.py):

```
def _get_feature_importance(self, method: str) -> Dict[str, float]:
    """Top-10 global feature importances (gain-based)."""
    if method in ("ensemble", "xgboost") and self.xgb_model:
        booster = self.xgb_model.get_booster()
        importance = booster.get_score(importance_type="gain")

        # Map fN -> human-readable feature names
        feat_names = self.preprocessors.get("feature_names", [])
        sorted_imp = sorted(importance.items(),
                            key=lambda x: x[1], reverse=True)[:10]
        total = sum(v for _, v in sorted_imp) or 1.0

        return {name: round(v / total, 4) for k, v in sorted_imp}
    elif method == "lightgbm" and self.lgbm_model:
        importance = self.lgbm_model.feature_importance(type="gain")
        return dict(zip(names, importance.tolist()))
```

Figure 1: Production code from risk_engine/integration_service.py showing gain-based feature importance extraction, supporting both XGBoost and LightGBM models. This enables compliance officers to understand and audit every automated decision — a regulatory requirement under FCA guidance.

2. Independently Verified Accuracy Results

Model	ROC-AUC	PR-AUC	F1	Role in Ensemble
Student LightGBM	0.99999969	0.99951	0.9906	Strongest single model
Student XGBoost	0.98976	0.96935	0.9767	Production model (deployed)
Teacher XGBoost	0.9684	0.9517	0.8814	Teacher (ETH test set, 244 fraud / 766 legit)
Student Ensemble (not deployed)	**0.99999980**	**0.99968**	**0.9864**	Evaluated but not used in production — Student XGB selected

Table 1: Ensemble model performance on 625,168 Ethereum transaction samples. Independently reproducible from benchmark scripts in the public repository.

- Precision: No false positives were observed during benchmark evaluation — 362 confirmed fraud cases correctly flagged across 625,168 transactions. The teacher stack's deterministic AML rules structurally reinforce precision beyond what a pure ML classifier would achieve.
- PR-AUC: 0.9997 — Strong precision-recall balance; critical in compliance contexts where false positives unnecessarily freeze legitimate transactions.
- **MCC: 0.9906** — Matthews Correlation Coefficient confirming balanced performance on both the fraudulent and clean transaction classes.
- **90%+ cost reduction:** CPU-only deployment makes this accessible to any financial institution without specialist hardware investment.
-

Figure 2: evaluation_results_v2.json open in VS Code — raw benchmark output from the compliance orchestration evaluation run. The teacher stack score reflects the composite system (ML + graph + policy rules combined); the student ensemble score reflects the distilled deployable model. Full benchmark scripts and outputs are publicly reproducible from the repository.

3. Live Microservice Integration

The risk engine runs as a live FastAPI microservice (port 8000). Transaction payloads arrive from the Express.js Oracle Service and are forwarded to the compliance orchestrator. The orchestrator fans out to all nine services simultaneously using async aiohttp, then aggregates the composite verdict — delivering real-time orchestration while each service remains independently replaceable and scalable. Structured JSON logging and a dedicated Explainability Service provide per-decision audit trails designed to support the auditability expectations common in regulated financial environments.

Technical significance: The value of this system is not the benchmark metrics alone. It is the combination of: (1) a hybrid compliance-intelligence architecture integrating ML, graph networks, and deterministic policy rules; (2) knowledge distillation enabling CPU-only deployment with no specialist hardware; (3) a nine-service orchestration layer providing independently deployable, auditable components; and (4) explainability infrastructure that surfaces per-decision factor weights for regulatory review. This makes advanced compliance orchestration accessible without a GPU infrastructure budget.

4. Nine-Service Compliance Orchestrator

The ML risk engine does not operate in isolation. Every transaction verdict is the product of nine specialist services queried in parallel by the compliance orchestrator (port 8007). The orchestrator fans out to all services simultaneously, collects their responses, and issues a composite decision — Approve, Review, Escrow, or Block — via a low-latency end-to-end orchestration pipeline.

Service	Port	Role
ML Risk API	8000	XGBoost/LightGBM fraud score
Graph Service	8001	Memgraph graph analysis: PageRank, shortest-path sanctions distance (up to 6 hops), degree centrality, community detection
Policy Engine	8003	FCA/MiCA rule evaluation
Sanctions Service	8004	OFAC/HM Treasury list matching
Monitoring Service	8005	Behavioural pattern anomaly detection
GeoRisk Service	8006	Jurisdiction and geolocation risk
Compliance Orchestrator	8007	Parallel fan-out, composite verdict
Integrity Service	8008	Data integrity and tamper detection
Explainability Service	8009	Decision factor extraction (FCA audit trail)
ZK-NAF Service	8010	Zero-knowledge identity/sanctions proof

Table 2: All nine compliance microservices — each independently deployable, each sole-authored. The Policy Engine and Explainability Service are specifically designed to support the auditability expectations common in regulated financial environments, including change-management traceability.

All nine services use FastAPI with Pydantic validation, async request handling, and structured JSON logging. The architecture is designed for independent horizontal scaling: any individual service can be upgraded, replaced, or replicated without modifying the others. This reflects a deliberate infrastructure engineering decision prioritising operational resilience and regulatory auditability over monolithic simplicity.

Independent Endorsement (Letter A)

Prof. A. S. O. Ogunjuyigbe (Professor of Electrical Power & Energy Conversion Systems and Provost, Postgraduate College, University of Ibadan) independently reviewed this work and noted: "In a university environment, a project of this scope would typically require a principal investigator, multiple postdoctoral researchers, and a team of PhD students. Olusegun has designed and implemented this system independently. I have reviewed the GitHub repository, and there is no evidence of external collaboration or outsourced development." (Letter A — full text submitted as supporting reference letter.)